



INSTITUTO FEDERAL

São Paulo

Câmpus Bragança Paulista

Tecnologia em Análise e Desenvolvimento de Sistemas

Desenvolvimento Web Back-end (BRADWBK)

Prof. Luiz Gustavo Diniz de Oliveira Vêras

Orientações para o Desenvolvimento de Projetos Web Back-end

Introdução

Este documento descreve as especificações acerca do projeto a ser desenvolvido como requisito da disciplina de Desenvolvimento Web Back-end (BRADWBK). O foco é a construção de uma **API RESTful** com servidor web integrado a banco de dados e comparações de desempenho em ambiente com e sem balanceamento de carga.

Objetivo do Trabalho

Desenvolver competências técnicas no desenvolvimento de APIs RESTful robustas, com banco de dados relacional e implantação em ambiente com balanceamento de carga. O projeto também visa desenvolver habilidades de teste funcional e de desempenho com ferramentas especializadas.

Dinâmica do Trabalho

O sistema deverá ser desenvolvido por grupos de **1 até 5 alunos**.

Cada grupo deverá:

- Escolher um domínio (tema da aplicação);
- Definir pelo menos **4 entidades** e envolvê-las em pelos menos um **relacionamento Um-para-muitos** e um **Muitos-para-Muitos**;
- Modelar e implementar uma API REST seguindo especificações HTTP. A API modelada deve considerar operações de CRUD (CREATE-READ-UPDATE-DELETE) para cada uma das entidades.

- Obrigatoriamente, todas as operações de CRUD devem ser implementadas para pelo menos duas entidades. Isso não é obrigatório para as outras duas, mas as mesmas devem estar envolvidas pelo menos uma vez em operações do tipo na API.
- Configurar autenticação para acesso à API utilizando JSON Web Tokens (JWT) com armazenamento de senha criptografada;
- Implantar a API em ambiente com **NGINX** configurado como balanceador de carga;
- Realizar testes de performance com **JMeter**, comparando os resultados com e sem o balanceamento de carga, de acordo com o que foi apresentado em aula;
- Realizar testes funcionais das rotas da API REST com ferramentas como **JMeter**, **Postman**, **Thunder Client**, **Insomnia**, entre outras.

Documento de Especificação da API

Cada grupo deverá elaborar um documento simples de especificação da API, contendo as seguintes informações:

- Domínio de aplicação da API (se for o mesmo de outra disciplina do curso, pode repeti-la aqui);
- Integrantes do grupo (nome e prontuário);
- Quatro entidades e a descrição do relacionamento entre elas;
- Tabela de modelagem da API REST, conforme exemplo:

Verbo HTTP	Path	Body de Requisição (se houver)	Body de Retorno (se houver)	Status HTTP de sucesso	Status HTTP de erro

Obs: A Tabela anterior pode ser substituída por modelagem em ferramentas como Swagger, OpenAPI ou similar.

Tecnologias

As seguintes tecnologias são preferíveis, mas **podem ser substituídas**, desde que mantenham a aderência aos requisitos de projeto definidos aqui e a arquitetura REST:

- **Back-end:** Java com Spring Boot e Spring Web;

- **Banco de Dados:** PostgreSQL, MySQL ou outro banco relacional. Se preferir por utilizar o banco de dados H2, que funciona de forma similar ao SQLite, porém é para Java;
- **ORM:** JPA/Hibernate para operações com banco de dados relacional;
- **Autenticação:** JWT no Spring e criptografia de senha
- **Balanceamento de Carga:** Servidor NGINX;
- **Teste de Performance:** Apache JMeter;
- **Teste Funcional:** JMeter, Postman, Thunder Client, Insomnia ou similar;
- **Versionamento de Código:** Git (preferencialmente GitHub, GitLab ou Bitbucket).

Entregas

O grupo deverá entregar:

- Documento de especificação do domínio, entidades e API Web em PDF ou link de documento hospedado em serviço na nuvem (Figma, Canva, etc);
- Código-fonte da API REST com suporte a CRUD das entidades em repositório Git;
- Scripts de configuração do NGINX para balanceamento de carga em repositório Git;
- Relatório comparativo de testes de performance com e sem balanceamento (importante duplicar o servidor SGBD), usando JMeter em PDF;
 - Se achar mais conveniente, pode rodar containers no Docker a partir da imagem do servidor SGBD. A imagem Docker do MySQL, por exemplo, é https://hub.docker.com/_/mysql/.
- Evidência dos testes funcionais com alguma ferramenta especializada. Podem ser geradas imagens JPEG ou PNG anexadas ao documento PDF do item anterior.

Data de Entrega de todos os documentos, códigos e apresentação em vídeo:
17/06/2026.

Algumas orientações adicionais para a apresentação:

- A documentação, relatórios e testes poderão ser agregadas em slides e entregue no dia;
- É necessário que o funcionamento do *back-end* seja demonstrado. Para tal, o grupo pode gravar vídeo para uso no dia da apresentação;
- O tempo máximo de vídeo para apresentação por grupo será de até **15 minutos**;

- Todos os membros do grupo devem falar no vídeo, caso contrário terão a nota obtida pelo grupo reduzida em **30%**. O aluno deve se identificar quando iniciar sua fala. Não é necessário que o estudante apareça no vídeo.

Critérios de Avaliação

Código do Critério	Descrição	Pontuação Máxima
CR1	Documento de especificação do domínio, entidades e da API Web	1,0
CR2	CRUD da entidade 1 pela API Web integrada ao Banco de Dados com ORM	1,0
CR3	CRUD da entidade 2 pela API Web integrada ao Banco de Dados com ORM	1,0
CR4	Alguma operação de CRUD com a entidade 3 pela API Web integrada ao Banco de Dados com ORM	1,0
CR5	Alguma operação de CRUD com a entidade 4 pela API Web integrada ao Banco de Dados com ORM	1,0
CR6	Implementação de 1 relacionamento Um-para-Muitos	0,5
CR7	Implementação de 1 relacionamento Muitos-para-Muitos	0,5
CR8	Autenticação funcional com pelo menos as rotas /auth/register (criar conta), auth/login (gerar token com a conta) e auth/logout (sair do acesso à API).	1,0
CR9	Configuração e funcionamento do balanceamento com NGINX	1,0
CR10	Relatório de testes de performance com JMeter	1,0
CR11	Testes funcionais documentados com Thunder Client, Postman ou equivalente	1,0
Total da pontuação		10,0

Um ótimo desenvolvimento a todos!